

Binary Search Trees (Part II) - Tree Searching Algorithms

Purpose

- Understanding the theory and usage of Searching algorithms for trees.
- Understanding the implementations of tree searching algorithms

Theory

- The searching for a particular node of a tree starts from the root and explores throughout the tree until the node is found (*goal node*).
- There can be more than one goal node within a tree in two different branches. (eg: inserted as key value pairs and search is done for a part of the value).
- For some problems any goal node is acceptable but for other problems, a minimum-depth goal node should be found (goal node closest to the root of the tree).
- There are two main searching algorithms for trees. Namely,
 - Depth first search (DFS)
 - Breadth first search (BFS)
- **Depth First Search (DFS)**
 - Searching starts from the root.
 - Explores a path all the way to a leaf before *backtracking* and exploring another path.
 - Similar to pre-order traversal but instead of using recursion another separate data structure is used for *backtracking*
 - A stack can be used in order to store the other nodes of a particular level for *backtracking*.
 - This process executes until the first goal node is found or the search is exhausted.

◦ DFS Algorithm

Put the root node on a stack;

While (stack is not empty) {

Remove a node from the stack;

if(removed node is a goal node){

return success;

}

Put all the children of the removed node into the stack;

}

return failure;

-
- Example,

● **Breadth First Search (BFS)**

- Searching starts from the root
- Explores the nodes nearest to the root before exploring nodes further away.
- A Queue can be used in order to store the child nodes of nearest nodes.
- This process executes until the first goal node is found or the search is exhausted.

○ **BFS Algorithm**

Put the root node on a queue;
while (queue is not empty) {

remove a node from the queue;

if (removed node is a goal node) {
return success;

}

put all children of the removed node onto the queue;
}
return failure;

● **BFS vs DFS**

- BFS finds the minimum-depth goal node first while in DFS the found goal node may necessarily be the minimum-depth node.
- For a large tree, number of nodes at each level increases exponentially with depth
- Therefore, memory requirements for BFS for a large tree may be excessive.
- On the other hand, for a large tree DFS may take an excessive long time in order to find even a very shallow depth goal node.

Question 01

1. Construct a binary search tree (BST) that can be used to store a sequence of **characters** that is obtained by a user. Assume that a key can occur at more than one and the value of the second encounter of the key is considered to be less than first encounter.

- a. Get an item from the user and search the level/levels for that item.
- b. Get an item from the user and search the first occurrence/ level according to the two searching algorithms and print the searching path (BFS & DFS).

Ex:

Inserted Items: **N O N L I N E A R D A T A T Y P E**

Search Character: **E**

Found on level 4

Search character: **T**

BFS: N->N->O->L->R->I->N->P->T

DFS: N->N->L->I->E->A->A->A->D->E->N->O->R->P->T